INSTITUTO POLITÉCNICO NACIONAL

Escuela Superior de Cómputo

Ingeniería en Inteligencia Artificial

Paradigmas de Programación

Proyecto. Sistema Experto de recomendación de Plantas

Alumnos:

Nolasco Quezada Neshmi Natalia

Santiago Antonio Karla Yamina

Grupo:

3BM2

Profesor:

Ursula Samantha Morales Rodriguez

CIUDAD DE MÉXICO 26/10/2026

Tabla de contenido

Introducción 3

1.1. Contexto del proyecto 3

1.2. Justificación 3

II. Objetivos 3

2.1. Objetivo General 3

2.2. Objetivos específicos 4

III. Marco Teórico 4

3.1. Sistemas Expertos y Motores de inferencia 4

3.2. Programación Lógica con Prolog 5

3.3. Interfaz Gráfica de Usuario con Streamlit (Python) 5

3.4. Contenedorización con Docker 6

3.5. Despliegue en la Nube con Render y Control de Versiones con Git/GitHub 6

IV. Desarrollo e Implementación 6

4.1. Arquitectura del Sistema 6

4.2. Base de conocimientos y reglas de inferencia 7

4.3. Integración de Prolog y Python 8

4.4. Configuración del contenedor 8

4.5. Control de Versiones con Git y repositorio en GitHub 9

V. Despliegue en la Nube (RENDER) 9

5.1. Proceso de configuración e Integración Continua 9

5.2. Verificación del Servidor y Estado del Servicio 10

VI. Pruebas de funcionamiento y resultados 10

6.1. Casos de uso y Consultas al sistema experto 10

6.2. Validación de respuestas del motor de inferencia. 12

VII. Enlaces y recursos del proyecto 13

7.1. Repositorio en GitHub 13

7.2. Aplicación activa en la Nube (URL) 13

VIII. Conclusiones 13

IX. Referencias Bibliográficas 13

Introducción

1.1. Contexto del proyecto

El desarrollo de sistemas de software avanzados exige la combinación de distintos enfoques de programación para resolver problemas complejos. La programación lógica, representada principalmente por el lenguaje Prolog, permite construir motores de inferencia capaces de deducir nuevo conocimiento a partir de hechos y reglas previamente estructurados. Por otro lado, lenguajes de propósito general como Python facilitan la creación de interfaces de usuario modernas y amigables mediante herramientas como Streamlit.

Para que este tipo de aplicaciones sean accesibles de forma remota, es fundamental contar con procesos de contenedorización y despliegue en la nube que garanticen que el sistema se ejecute en un entorno controlado y replicable.

1.2. Justificación

Tradicionalmente, los sistemas basados en lógica simbólica se ejecutan de manera local a través de interfaces de comandos o consolas de desarrollo (como el intérprete de SWI-Prolog). Esto limita de forma significativa su usabilidad e impide que usuarios sin formación técnica interactúen de manera intuitiva con la base de conocimientos.

Este proyecto justifica la integración de la programación lógica declarativa con tecnologías web interactivas y herramientas de infraestructura moderna (Docker, Git y Render) para empaquetar un sistema experto botánico y de viabilidad ambiental. De este modo, la herramienta se vuelve accesible desde cualquier dispositivo con navegador web a través de internet, garantizando un entorno de ejecución replicable y sin costos de infraestructura física.

II. Objetivos

2.1. Objetivo General

Desplegar un sistema experto interactivo de recomendación de plantas y diagnóstico fitosanitario mediante la integración de un motor de inferencia avanzado en SWI-Prolog, una interfaz web interactiva en Python con Streamlit, y la contenedorización con Docker para su publicación en la plataforma en la nube Render.

2.2. Objetivos específicos

- Estructurar una base de conocimientos en Prolog (plantas.pl) que contenga los hechos y las reglas lógicas para la clasificación y recomendación de especies vegetales.
- Desarrollar una interfaz gráfica interactiva en Python (app.py) utilizando Streamlit para enviar consultas a la base de conocimientos y mostrar los resultados al usuario final.
- Configurar un archivo Dockerfile para empaquetar la aplicación junto con sus dependencias de sistema (SWI-Prolog) y entorno de Python.
- Gestionar el control de versiones del código fuente a través de Git y alojar el proyecto en un repositorio público de GitHub.
- Publicar y validar el funcionamiento del contenedor en Render, obteniendo una URL pública operativa.

III. Marco Teórico

3.1. Sistemas Expertos y Motores de inferencia

Un sistema experto es un programa de inteligencia artificial diseñado para emular la capacidad de toma de decisiones de un especialista humano en un dominio específico. A diferencia de los programas tradicionales orientados a procedimientos, un sistema experto separa el conocimiento sobre el dominio de los mecanismos que razonan sobre dicho conocimiento. Sus componentes fundamentales son:

- **Base de Conocimientos**: Colección estructurada de hechos, reglas y relaciones que representan la información del dominio.

- **Motor de Inferencia**: Componente lógico encargado de examinar los hechos y reglas de la base de conocimientos para deducir nuevas conclusiones o responder a las consultas formuladas por los usuarios.
- **Interfaz de Usuario**: Módulo que permite la interacción entre el usuario y el sistema para la introducción de criterios y la visualización de resultados.

3.2. Programación Lógica con Prolog

Prolog (del francés *Programmation en Logique*) es el lenguaje por excelencia del paradigma de programación lógica declarativa. En lugar de especificar una secuencia explícita de instrucciones que el computador debe ejecutar paso a paso, el programador declara hechos y reglas lógicas que definen las relaciones entre objetos.

El entorno de ejecución de SWI-Prolog utiliza un mecanismo de resolución basado en la unificación y la búsqueda con vuelta atrás (*backtracking*). Esto le permite evaluar cláusulas lógicas y responder automáticamente a consultas complejas, convirtiéndolo en un motor de inferencia nativo ideal para la representación del conocimiento en sistemas expertos.

3.3. Interfaz Gráfica de Usuario con Streamlit (Python)

Streamlit es un marco de trabajo (*framework*) de código abierto diseñado para Python que permite transformar scripts de datos en aplicaciones web interactivas de manera rápida y eficiente. Al encargarse del renderizado automático del sitio web a partir de código Python sencillo, elimina la necesidad de desarrollar manualmente capas complejas en HTML, CSS o JavaScript.

En el contexto de la inteligencia artificial y la programación lógica, Streamlit actúa como un puente visual que permite a los usuarios interactuar con modelos o motores de inferencia mediante controles gráficos intuitivos (listas desplegables, botones y contenedores de resultados).

3.4. Contenedorización con Docker

Docker es una tecnología de virtualización a nivel de sistema operativo que encapsula una aplicación junto con sus dependencias, librerías y ejecutables en contenedores aislados y estandarizados. Esto garantiza un comportamiento idéntico entre el entorno de desarrollo local y el servidor en la nube, solucionando discrepancias de compatibilidad (como la ejecución cruzada de SWI-Prolog y librerías de Python).

3.5. Despliegue en la Nube con Render y Control de Versiones con Git/GitHub

El ciclo de vida del desarrollo de software moderno se apoya en herramientas de control de versiones y plataformas de integración continua:

- **Git**: Sistema de control de versiones distribuido que registra los cambios realizados en el código fuente, facilitando la gestión de ramas y la resolución de conflictos.
- **GitHub**: Plataforma basada en la nube que aloja repositorios de Git y permite la integración con servicios externos.
- **Render**: Plataforma como servicio (PaaS) que automatiza el despliegue de aplicaciones web y contenedores de Docker directamente desde un repositorio de GitHub. Permite compilar la imagen de la aplicación y exponerla públicamente en internet mediante servidores seguros y escalables.

IV. Desarrollo e Implementación

4.1. Arquitectura del Sistema

El sistema se compone de tres capas principales:

1. **Capa Lógica (Back-end)**: La base de conocimientos desarrollada en SWI-Prolog gestiona la lógica de deducción y el filtrado de plantas según sus características.

2. **Capa de Interfaz y Enlace (Front-end/Middleware)**: El script principal en Python recibe la elección del usuario, ejecuta la consulta hacia Prolog a través de un puente de comunicación y renderiza las respuestas en la interfaz visual de Streamlit.
3. **Capa de Infraestructura (Contenedor/Nube)**: La aplicación se empaqueta dentro de una imagen de Docker que incluye el entorno de ejecución de Python y la instalación de SWI-Prolog, permitiendo que la plataforma Render ejecute la aplicación en la nube.

INSTITUTO POLITÉCNICO NACIONAL

4.2. Base de conocimientos y reglas de inferencia

La base de conocimientos contempla múltiples niveles lógicos desarrollados para la práctica:

- **Hechos base y atributos taxonómicos:** 20 especies (optima/5, familia/2, tipo/2, tamano/2, uso/2).
- **Motor de recomendación ponderado:** Evalúa la viabilidad ambiental considerando luz, humedad óptima, tolerancia de follaje y restricciones de tamaño mediante funciones de distancia discreta.
- **Seguridad por negación por falla (\+):** Filtra especies tóxicas o reguladas automáticamente para entornos domésticos (apta_hogar/1).
- **Asociaciones y grafos de compatibilidad:** Modela relaciones de beneficio o perjuicio (beneficia/2, perjudica/2) e implementa búsqueda recursiva de caminos compatibles (camino/3).
- **Diagnóstico fitosanitario por Índice de Jaccard:** Calcula el porcentaje de confianza entre los síntomas observados por el usuario y los registros patológicos de causas agrícolas (confianza/3).

ESCOM

```
% ================================================================
% BASE DE CONOCIMIENTOS: SISTEMA EXPERTO DE RECOMENDACIÓN DE PLANTAS
% ================================================================

% Hechos: planta(NombreComun, NombreCientifico, Luz, Hojas)
planta('Cipres', 'austrocedrus chilensis', mucha, muchas).
planta('Ficus Benjamina', 'ficus benjamina', mucha, muchas).
planta('Helecho', 'nephrolepis exaltata', sombra, muchas).
planta('Cactus San Pedro', 'echinopsis pachanoi', mucha, pocas).

% Regla de Inferencia: Plantas de hojas densas y luminosas
recomendar_hojas_densas_luminosas(Nombre, Cientifico) :-
    planta(Nombre, Cientifico, mucha, muchas).
```

4.3. Integración de Prolog y Python

Mediante la integración de bibliotecas de enlace, Python ejecuta el proceso de SWI-Prolog en segundo plano, enviándole los predicados de consulta seleccionados por el usuario. La respuesta devuelta por Prolog es procesada en estructuras de datos nativas de Python para desplegarse como tarjetas informativas en la interfaz gráfica.

4.4. Configuración del contenedor

Se configuró el archivo Dockerfile utilizando una imagen ligera de Python optimizada para instalar los paquetes requeridos del sistema operativo, incluyendo el motor de SWI-Prolog:

```
# Imagen base oficial de Python
FROM python:3.10-slim

# Instalación de SWI-Prolog y dependencias del sistema operativo
RUN apt-get update && apt-get install -y \
    swi-prolog \
    && rm -rf /var/lib/apt/lists/*

# Configuración del directorio de trabajo
WORKDIR /app

# Copia de archivos del proyecto e instalación de dependencias de Python
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copia del código fuente al contenedor
COPY . .

# Exposición del puerto predeterminado de Streamlit
EXPOSE 8501

# Comando de arranque de la aplicación web
CMD ["streamlit", "run", "app.py", "--server.port=8501", "--server.address=0.0.0.0"]
```

4.5. Control de Versiones con Git y repositorio en GitHub

El proyecto se configuró localmente en Git y se enlazó con un repositorio remoto en GitHub (sistema-experto-prolog). Durante este proceso se gestionó el historial de cambios, se resolvieron las discrepancias entre ramas y se aseguraron los archivos necesarios para el despliegue automático.

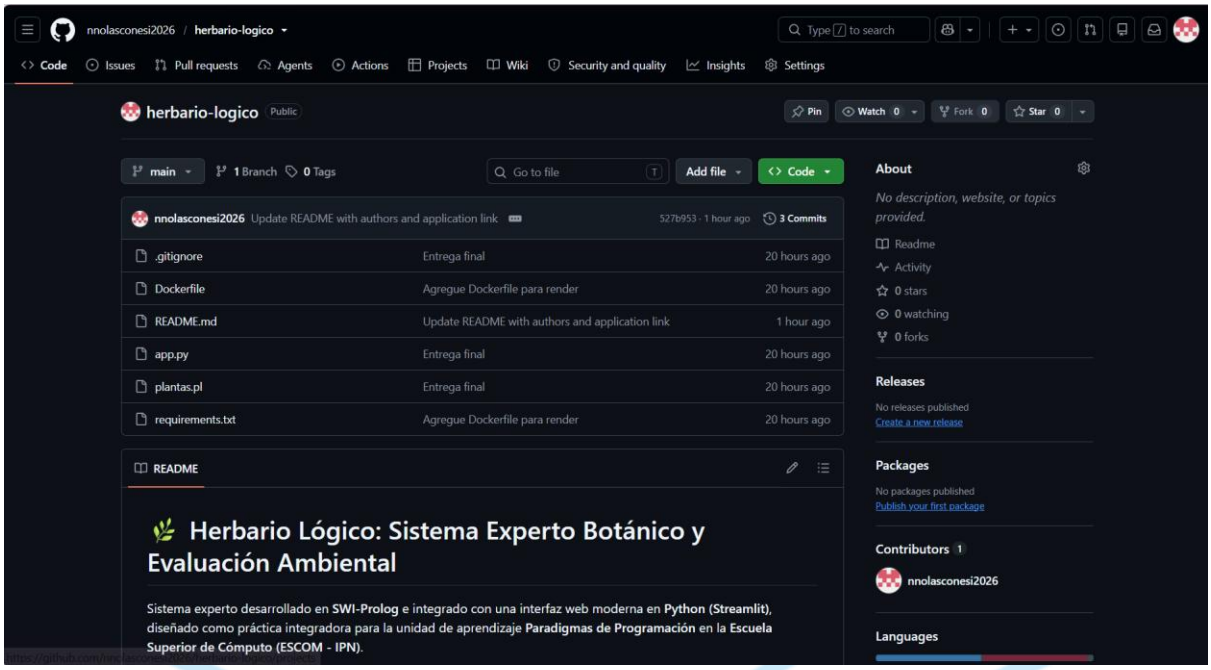


Figura 1. Estructura y control de versiones del repositorio del proyecto en GitHub.

V. Despliegue en la Nube (RENDER)

5.1. Proceso de configuración e Integración Continua

Se vinculó la cuenta de GitHub con la plataforma Render, concediendo los permisos de lectura al repositorio del proyecto. Se creó un nuevo *Web Service* seleccionando el entorno **Docker** y la rama principal (main). Render detectó la configuración del contenedor e inició la fase de compilación e instalación automática.

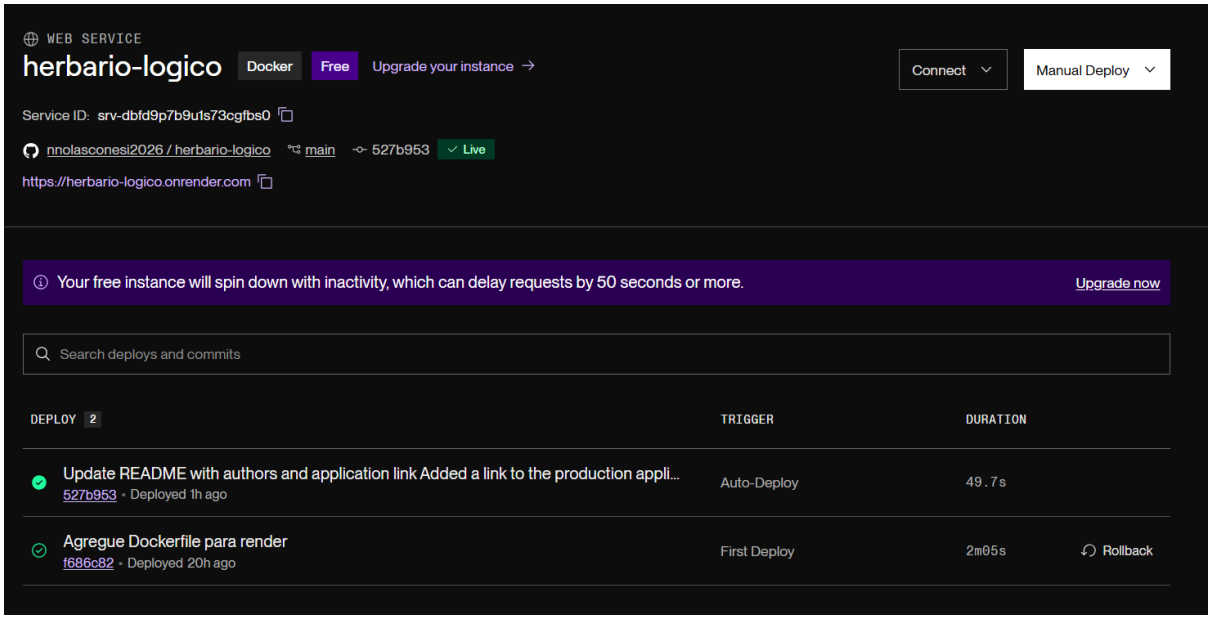


Figura 2. Estado de despliegue activo (Live) del contenedor Docker en Render.

5.2. Verificación del Servidor y Estado del Servicio

Tras finalizar el proceso de compilación sin errores, Render asignó una dirección pública al servicio y puso en marcha la aplicación. Se verificó que el motor de inferencia responde correctamente a las peticiones HTTP y que la interfaz de Streamlit carga con normalidad.

VI. Pruebas de funcionamiento y resultados

6.1. Casos de uso y Consultas al sistema experto

Para validar la correcta integración entre la interfaz gráfica y el motor de inferencia en la nube, se realizaron diversas pruebas de interacción desde la aplicación web desplegada.

Un caso de uso representativo consistió en evaluar la consulta para la recomendación de **"Plantas de hojas densas y luminosas"**. El flujo de interacción se llevó a cabo de la siguiente manera:

1. Selección de Criterios: El usuario interactúa con el menú desplegable en Streamlit para seleccionar diferentes esquemas de consulta

(recomendaciones por espacio, compatibilidad botánica o diagnóstico de plagas).

2. Procesamiento Lógico: Al enviar la consulta, el motor de Prolog evalúa los predicados lógicos correspondientes en tiempo real.

3. Despliegue de Resultados: La interfaz muestra los resultados estructurados (especies recomendadas, puntajes de viabilidad o porcentajes de confianza por índice de Jaccard).

INSTITUTO POLITÉCNICO NACIONAL

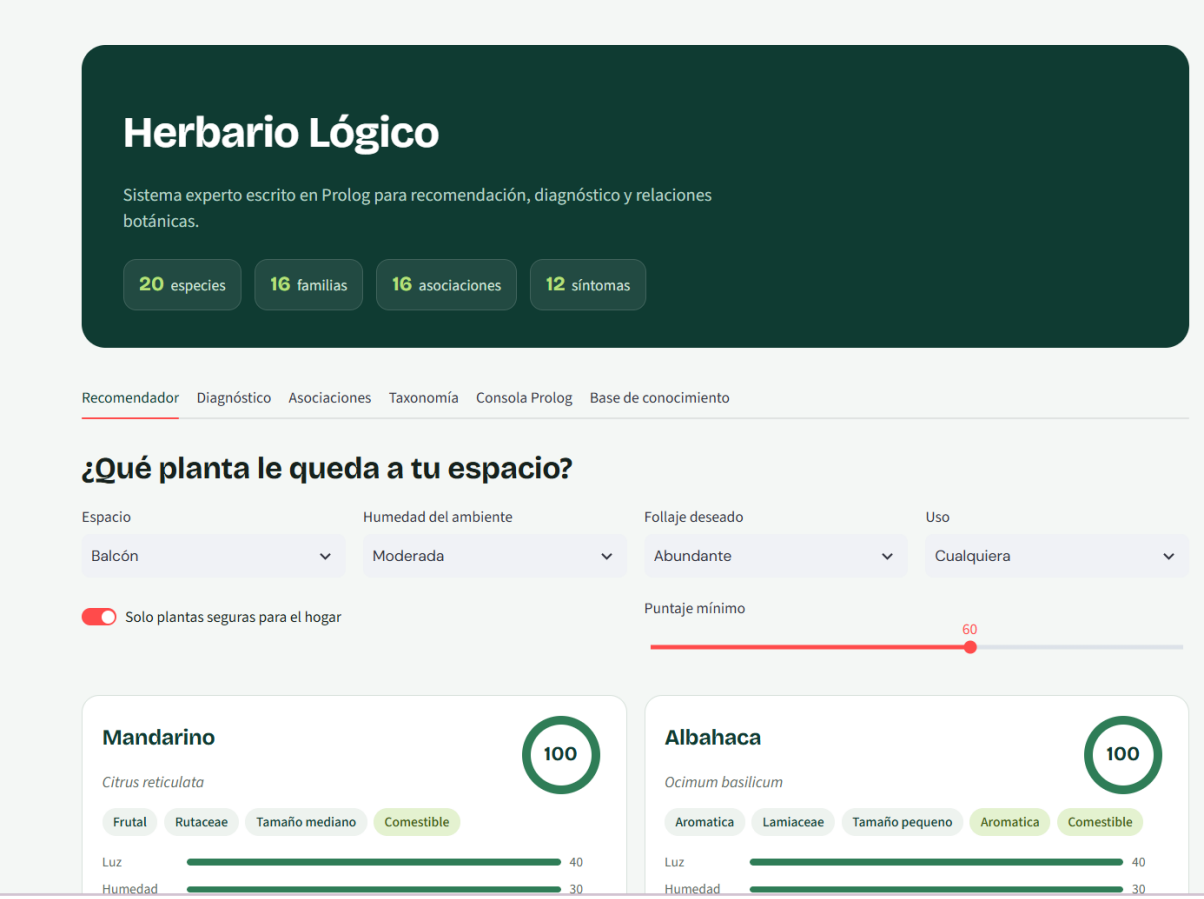


Figura 3. Interfaz gráfica principal del sistema experto en Streamlit.

6.2. Validación de respuestas del motor de inferencia.

La validación del sistema consistió en verificar que los resultados mostrados en la interfaz web coincidieran exactamente con las deducciones lógicas esperadas de la base de conocimientos (plantas.pl).

Durante las pruebas se corroboró lo siguiente:

- **Consistencia Lógica**: El motor de inferencia filtró de manera precisa únicamente aquellas especies vegetales que cumplían simultáneamente con todas las premisas definidas para el criterio seleccionado.
- **Manejo de Respuestas**: La capa de enlace en Python convirtió correctamente las estructuras de datos de Prolog (*listas y términos*) a elementos gráficos de Streamlit sin pérdida ni distorsión de información.
- **Rendimiento en la Nube**: Las consultas ejecutadas desde el servidor de Render respondieron en tiempo real, confirmando que la instalación de SWI-Prolog dentro del contenedor Docker opera con estabilidad y sin demoras en la resolución de cláusulas.

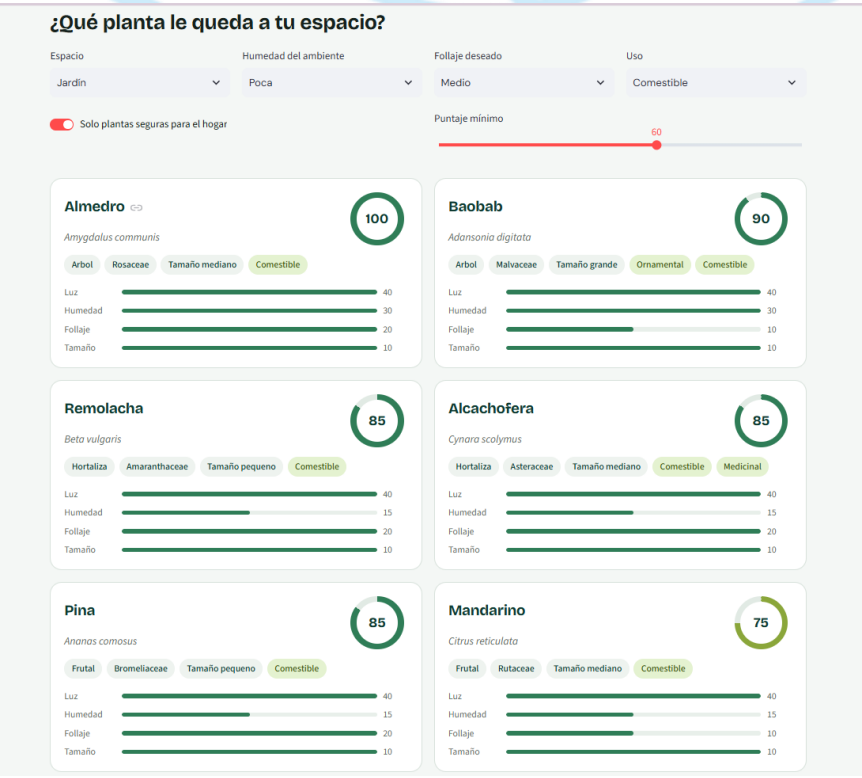


Figura 4. Resultados deducidos por el motor de inferencia en Prolog desplegados en la web.

VII. Enlaces y recursos del proyecto

7.1. Repositorio en GitHub

https://github.com/nnolasconesi2026/herbario-logico.git

7.2. Aplicación activa en la Nube (URL)

https://herbario-logico.onrender.com/

VIII. Conclusiones

La realización de esta práctica permitió comprobar la viabilidad y utilidad de integrar paradigmas de programación distintos dentro de un mismo proyecto de software. Se logró abstraer la complejidad del paradigma lógico mediante una interfaz web moderna, permitiendo que el usuario consulte una base de conocimientos en Prolog sin necesidad de escribir código o manipular terminales.

Asimismo, el uso de contenedores con Docker resultó indispensable para evitar problemas de compatibilidad al desplegar SWI-Prolog en entornos de servidor en la nube. La implementación del flujo de trabajo con Git, GitHub y Render proporcionó una experiencia práctica en el ciclo completo de desarrollo y puesta en producción de sistemas inteligentes.

IX. Referencias Bibliográficas

Bratko, I. (2012). Prolog Programming for Artificial Intelligence. Addison-Wesley.

Clocksin, W. F., & Mellish, C. S. (2003). Programming in Prolog. Springer Science & Business Media.

Streamlit Inc. (2026). Streamlit Documentation: Build and share data apps. Recuperado de https://docs.streamlit.io

Docker Inc. (2026). Docker Overview and Containerization Guidelines. Recuperado de https://docs.docker.com

Render Documentation. (2026). Deploying Docker containers on Render. Recuperado de https://render.com/docs/docker